

Student

I want to put everything together - design a complete warehouse from a business requirement, write analytics queries over it, understand the mistakes that trip up candidates, and practice the design interview format.

Claude • AI Assistant

This chapter is the synthesis. Every concept from Chapters 1 through 4 converges here into one complete workflow: requirements to Bus Matrix to logical model to physical DDL to analytics queries.

The chapter then does what no other chapter does - it shows you the mistakes that lose interviews, and it drills the verbal whiteboard skill that distinguishes a candidate who knows dimensional modeling from one who can think through it live under pressure. Five subsections:

1. **End-to-end warehouse design** - from business requirement to physical model in one exercise.
2. **Sales analytics** - revenue, trend, and cohort queries over the star schema.
3. **Customer analytics** - RFM segmentation, retention, and LTV queries.
4. **Common modeling mistakes** - the traps that appear in real designs and in interviews.
5. **Mock design interview** - the whiteboard protocol, the verbal discipline, and five practice questions.

CONCEPT CARD

The four-step Kimball design process - the spine of this chapter:

1. **Select the business process** - identify the organizational activity being modeled: 'retail point-of-sale transactions.'
2. **Declare the grain** - write one precise English sentence: 'One row represents one line item on one POS transaction.'
3. **Identify the dimensions** - list every FK the fact table needs: Date, Store, Product, Customer, Employee, Promotion.
4. **Identify the facts** - list the measures consistent with the grain: *quantitysold*, *unitprice*, *discountamount*, *netrevenue*, *tax_amount*. Classify each as additive, semi-additive, or non-additive.

SUMMARY CARD

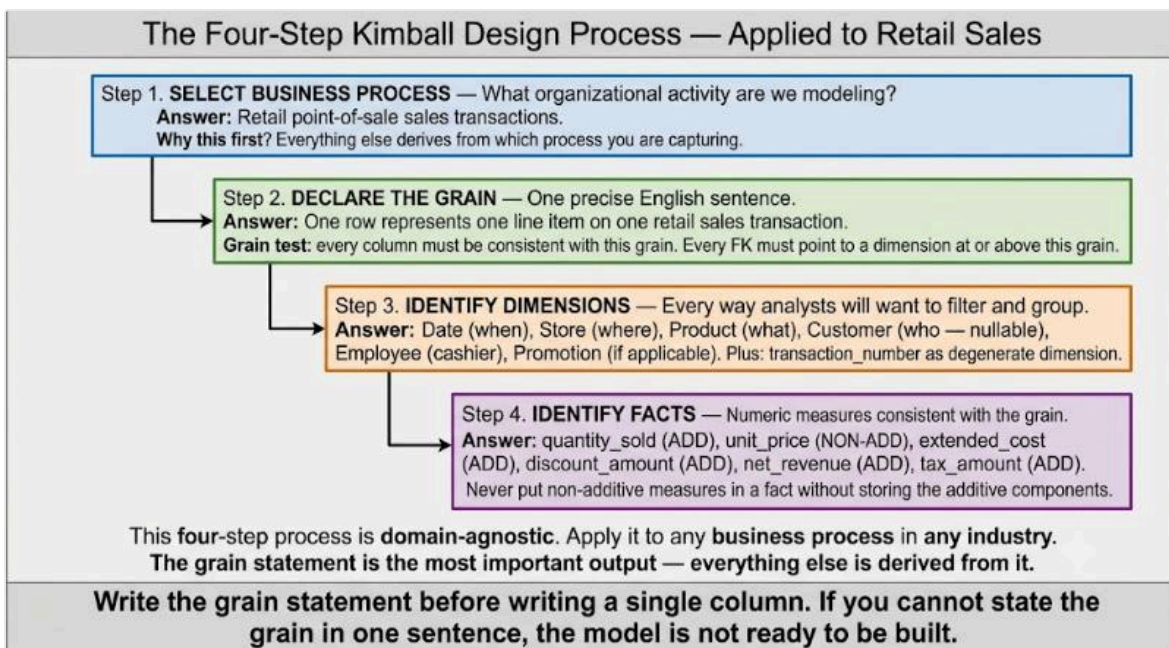
By the end of this chapter you will be able to:

- Execute the four-step Kimball design process for any business domain from scratch.
- Write production-quality SQL for revenue trend, cohort, RFM, retention, and LTV analysis.
- Identify and fix the six most common dimensional modeling mistakes.
- Articulate a dimensional design aloud in the whiteboard interview format: grain → dimensions → facts → SCD decision → Bus Matrix position.
- Answer the five most common dimensional modeling interview questions with structured, complete responses.

🔮 AI DEEP DIVE PROMPT

Warm-up: **"You are interviewing for a data engineering role. The interviewer says: 'Design a data warehouse for a grocery chain.' Walk me through your complete response - from clarifying questions through grain statement, dimension list, fact table DDL, Bus Matrix, and one example analytics query.**

Time yourself: you should complete this in under 8 minutes."



The Four-Step Kimball Design Process - Applied to Retail Sales

Six Common Dimensional Modeling Mistakes - Symptom, Root Cause, and Fix

Mistake	Symptom	Root cause	Fix
Mixed grain in one fact	SUM(revenue) doubles or triples unexpectedly	Two different grains (e.g., header + line) in one table	Separate fact tables, one per grain statement

Mistake	Symptom	Root cause	Fix
Over-snowflaking	3–5 JOIN hops to reach a dimension attribute	Normalizing dimension into parent tables	Flatten all hierarchy levels into one dimension row
Missing conformed dimensions	Cannot join Sales mart to Inventory mart by product	Each mart built its own product dimension with own keys	Establish conformed dim_product before building either mart
Natural keys as fact FKs	Warehouse breaks when source system rennumbers keys	No surrogate key; FK is the source natural key	Surrogate key pipeline; facts always use warehouse surrogate
Non-additive measure SUMmed	Profit margin % report shows 1,200%	SUM(margin_pct) across rows; not volume-weighted	Store additive numerator/denominator; derive ratio at query time
NULL FK in fact table	GROUP BY customer_key drops 30% of revenue	Anonymous customers have NULL FK, excluded by JOIN	Unknown Customer surrogate (e.g., -1); no NULLs in fact FKs

sql – Complete physical DDL – retail star schema from the end-to-end design exercise

```
-- =====
-- CONFORMED DIMENSIONS
-- =====

CREATE TABLE dim_date (
    date_key      INT  PRIMARY KEY,    -- YYYYMMDD
    calendar_date DATE NOT NULL,
    day_name      VARCHAR(10), is_weekend TINYINT, is_holiday TINYINT,
    month_number  TINYINT, month_name VARCHAR(10),
    quarter_number TINYINT, year_number SMALLINT,
    fiscal_year   SMALLINT, fiscal_quarter TINYINT, fiscal_month TINYINT,
    relative_period_month INT DEFAULT 0
);

CREATE TABLE dim_store (
    store_key     INT PRIMARY KEY,
    store_code    VARCHAR(10) NOT NULL,
```

```

        store_name    VARCHAR(200),
        city VARCHAR(100), state VARCHAR(50),
        region VARCHAR(50), store_format VARCHAR(30)
    );

CREATE TABLE dim_product (
    product_key      INT PRIMARY KEY,
    product_code     VARCHAR(20) NOT NULL,
    product_name     VARCHAR(200) NOT NULL,
    brand            VARCHAR(100),
    subcategory_name VARCHAR(100),
    category_name    VARCHAR(100),
    department_name  VARCHAR(100),
    is_active        TINYINT NOT NULL DEFAULT 1
);

CREATE TABLE dim_customer (
    customer_key     INT PRIMARY KEY,
    customer_id      VARCHAR(20) NOT NULL,
    customer_name    VARCHAR(200) NOT NULL,
    email            VARCHAR(200),
    city             VARCHAR(100),
    loyalty_tier     VARCHAR(15),
    acquisition_ch    VARCHAR(50),
    first_purchase_dt DATE,
    row_start_date   DATE NOT NULL,
    row_end_date     DATE NOT NULL DEFAULT '9999-12-31',
    is_current       TINYINT NOT NULL DEFAULT 1
);

CREATE TABLE dim_promotion (
    promotion_key    INT PRIMARY KEY,
    promotion_name   VARCHAR(200),
    promotion_type   VARCHAR(50),
    discount_pct     DECIMAL(5,2),
    start_date       DATE, end_date DATE
);

-- =====
-- FACT TABLE
-- Grain: one row per line item on a retail sales transaction
-- =====

CREATE TABLE fact_sales_transaction (
    date_key         INT NOT NULL,
    store_key        INT NOT NULL,
    product_key      INT NOT NULL,

```

```

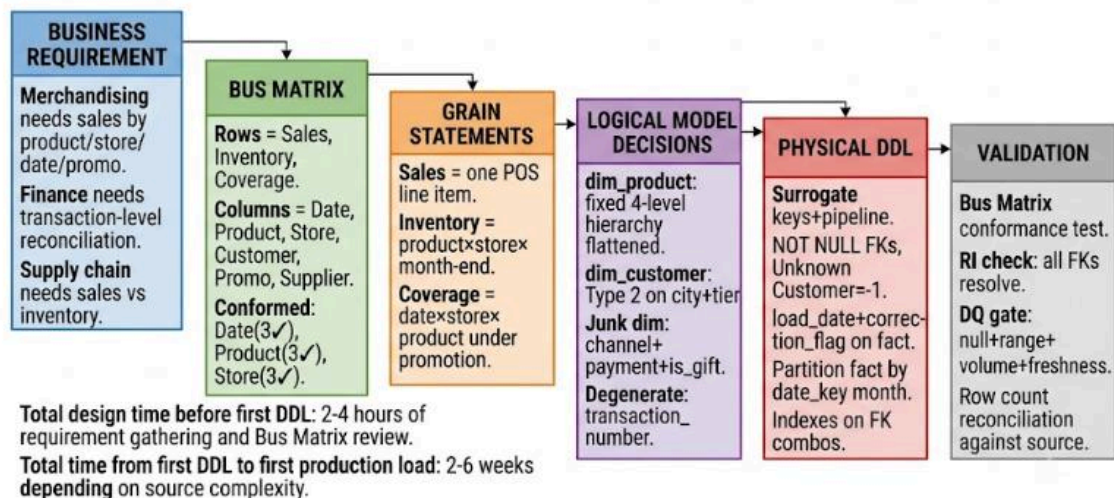
customer_key      INT NOT NULL DEFAULT -1, -- -1 = Unknown Customer
promotion_key     INT,                      -- NULL = no promotion
transaction_number VARCHAR(25) NOT NULL,    -- DEGENERATE DIMENSION
quantity_sold     INT NOT NULL,
unit_price        DECIMAL(10,2),           -- NON-additive
extended_cost     DECIMAL(12,2),
discount_amount   DECIMAL(10,2) DEFAULT 0,
net_revenue       DECIMAL(12,2) NOT NULL,
tax_amount        DECIMAL(10,2) DEFAULT 0,
load_date         DATE NOT NULL DEFAULT CURRENT_DATE,
correction_flag   TINYINT NOT NULL DEFAULT 0
);

```

END-TO-END WAREHOUSE DESIGN — FROM REQUIREMENT TO PHYSICAL MODEL

Claude • AI Assistant

End-to-End Design – From Requirement to Physical Model in Six Artifacts



Every artifact in this diagram traces back to a business question. If a column has no owning business question, it should not be in the model.

End-to-End Design - From Requirement to Physical Model in Six Artifacts

Student

Give me the complete design exercise - start from a business requirement and produce a ready-to-build physical model.

Claude • AI Assistant

The exercise follows the four-step Kimball process exactly. The business requirement comes first - everything the data model delivers must trace back to a business question. The design is complete when you can answer those questions from the physical model.

CONCEPT CARD

Step 0 - Understand the business requirements:

1. **Ask before designing** - the most common modeling mistake is building without requirements.

Three questions to ask: (a) What business decisions will this warehouse inform? (b) Who are the consumers - which teams, which tools?

(c) What is the finest level of detail needed (the grain)?

2. **The retail requirement** - 'Our merchandising team needs to analyze sales performance by product, store, date, and promotional period. Finance needs revenue reconciliation at the transaction level. Supply chain needs to compare sales velocity against inventory levels.'
3. **Business processes identified** - Sales Transactions, Inventory Snapshot, Promotional Coverage. Three processes → at least three fact tables.
4. **Grain decisions from requirements** - 'Transaction level' → grain: one row per POS line item. 'Compare sales velocity against inventory' → inventory grain: product × store × month-end.

CONCEPT CARD

Step 1 - Build the Bus Matrix:

1. **Rows** - Retail Sales Transactions, Monthly Inventory Snapshot, Promotional Coverage.
2. **Columns** - Date, Product, Store, Customer, Promotion, Supplier.
3. **Checkmarks** - Sales: Date ✓ Product ✓ Store ✓ Customer ✓ Promotion ✓. Inventory: Date ✓ Product ✓ Store ✓ Supplier ✓. Coverage: Date ✓ Product ✓ Store ✓ Promotion ✓.
4. **Conformed** - Date (3 checks), Product (3 checks), Store (3 checks). Build these three before any fact table.
5. **Process-specific** - Customer (Sales only), Supplier (Inventory only).

CONCEPT CARD

Step 2 - Logical model decisions:

1. **dim_product hierarchy** - fixed depth: product → subcategory → category → department. All four levels flattened into one dim_product row.
2. **dim_customer SCD** - Type 2 on: `city` , `loyalty_tier` , `email` . Type 1 on: `acquisition_channel` (no historical value).
3. **dim_date** - pre-built with fiscal calendar (April-to-March), holiday flags, `relativeperiodmonth` for rolling-window reports.
4. **Junk dimension** - `order_channel` (in-store/online/phone), `payment_method` (cash/card/UPI/voucher), `is_gift` (Y/N) → consolidated into `dim_order_junk` to narrow the fact table.
5. **Degenerate dimension** - `transaction_number` stays in the fact. No meaningful attributes; high identifying value for receipt-level queries.

CONCEPT CARD

Step 3 - Physical model checklist:

1. **Surrogate keys** - all dimensions use generated integer surrogate keys. Key pipeline table maps source natural keys.
2. **NOT NULL on FKs** - all fact FK columns are NOT NULL. Unknown Customer surrogate (-1) handles anonymous purchases.
3. **Correction metadata** - `load_date DATE` and `correction_flag TINYINT` on the fact table from day one.
4. **Partitioning strategy** - `fact_sales_transaction` partitioned by `date_key` (range partition). Each month is one partition. Queries filtered by date never scan historical partitions.
5. **Indexes** - fact table: composite index on (`storekey`, `datekey`), (`productkey`, `datekey`), (`customerkey`, `datekey`). Dimension tables: unique index on natural key for ETL lookup speed.

🔗 ANALOGY

The end-to-end design process is like **planning a city before laying the first brick**.

Requirements = the population's needs (transport, commerce, housing).

Bus Matrix = the city plan showing which districts share which infrastructure. Logical model = the district layout decisions.

Physical model = the engineering blueprints. Building without the plan produces a city that cannot grow without tearing itself apart.

Building with the plan produces a city where a new district plugs in without disrupting existing ones.

⚠️ WARNING

Premature physical optimization ruins logical designs. Do not make grain or dimension decisions based on 'what will be fast in our specific database.' Decide the grain from business requirements first. Optimize the physical implementation after the logical model is correct.

Many warehouses have wrong grain decisions that cannot be fixed without a full rebuild - because the designer optimized before they understood.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Starting the design with 'what tables do we have in the source system?' Source system structure is irrelevant to dimensional model design. An OLTP schema designed for write performance is not a starting point for an analytical model.

Correct Way: Start with 'what business questions must this answer?' Then: four-step process. Source system structure is consulted only in the ETL phase - to understand how to extract, not what to model.

EXAM TIP CARD

Interview classic: *"How do you approach designing a data warehouse from scratch?"* Winning answer: *"Four steps. First: clarify requirements - what decisions, who consumes, what granularity.*

Second: build the Bus Matrix - identify business processes, candidate dimensions, mark conformance. Third: for each process - declare the grain (one sentence), identify dimensions, identify and classify facts.

Fourth: physical decisions - surrogate keys, SCD types per dimension, partition strategy, correction metadata. Always requirements before DDL.

Always grain before columns."

🔥 EXERCISE

You are asked to design a warehouse for a ride-hailing company. Business processes: trip completion, driver earnings, rider complaints, and surge pricing events.

Apply the four-step process: (a) write the grain statement for each process, (b) build the Bus Matrix, (c) identify which dimensions are conformed, (d) classify three measures per fact table as additive, semi-additive, or non-additive.

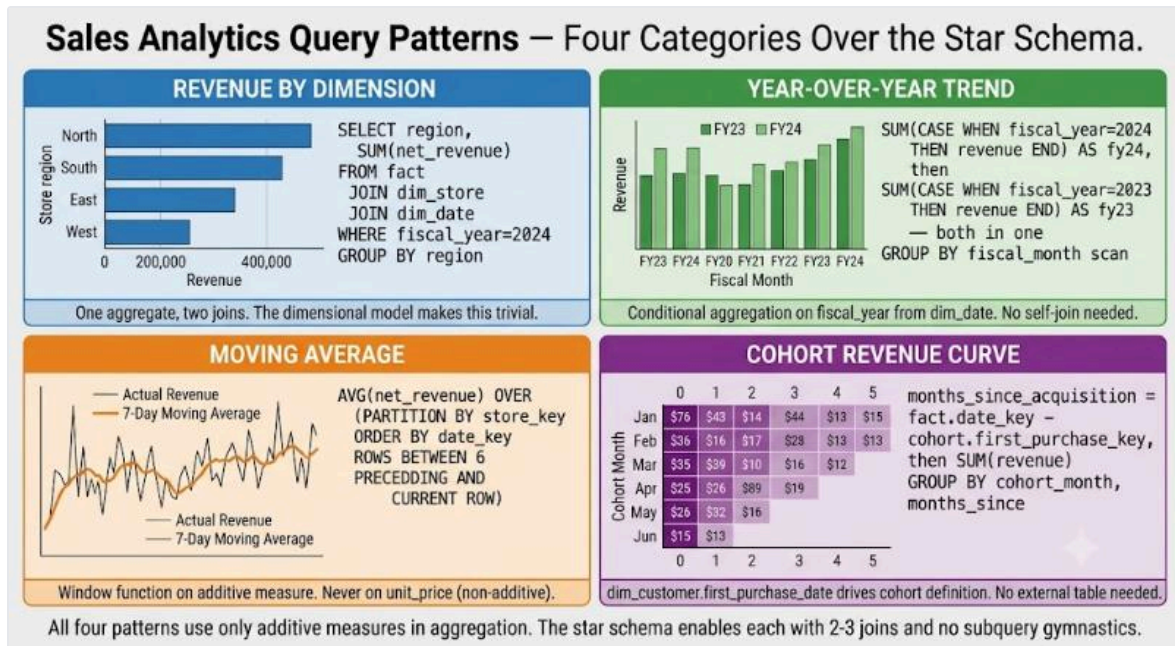
🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *"Apply the four-step Kimball design process to a hospital system with four business processes: outpatient encounters, inpatient admissions, pharmacy dispensing, and lab orders. For each step: list the business processes, declare a grain statement, identify dimensions (marking conformed ones), and classify all measures.*

Then build the Bus Matrix and identify which three dimensions to build first."

SALES ANALYTICS — REVENUE, TREND, AND COHORT QUERIES

Claude • AI Assistant



Sales Analytics Query Patterns - Four Categories Over the Star Schema

sql – Production sales analytics queries – trend, YoY, and cohort

```

-- =====
-- 1. MONTH-OVER-MONTH REVENUE WITH LAG
-- =====

WITH monthly_revenue AS (
    SELECT
        d.fiscal_year,
        d.fiscal_month,
        SUM(f.net_revenue) AS revenue
    FROM   fact_sales_transaction f
    JOIN   dim_date d ON f.date_key = d.date_key
    WHERE  d.relative_period_month BETWEEN -11 AND 0
    GROUP BY d.fiscal_year, d.fiscal_month
)
SELECT
    fiscal_year,
    fiscal_month,

```

```

        revenue,
        LAG(revenue, 1) OVER (ORDER BY fiscal_year, fiscal_month) AS
prior_month_revenue,
        ROUND(100.0 * (revenue - LAG(revenue,1) OVER (ORDER BY fiscal_year,
fiscal_month))
                / NULLIF(LAG(revenue,1) OVER (ORDER BY fiscal_year,
fiscal_month),0)
                , 1) AS
mom_growth_pct
FROM   monthly_revenue
ORDER BY fiscal_year, fiscal_month;

-- =====
-- 2. PROMOTION LIFT: revenue per unit during vs outside promo
-- =====
SELECT
    p.product_name,
    prm.promotion_name,
    SUM(CASE WHEN f.promotion_key IS NOT NULL
            THEN f.net_revenue ELSE 0 END) AS promo_revenue,
    SUM(CASE WHEN f.promotion_key IS NOT NULL
            THEN f.quantity_sold ELSE 0 END) AS promo_units,
    SUM(CASE WHEN f.promotion_key IS NULL
            THEN f.net_revenue ELSE 0 END) AS base_revenue,
    SUM(CASE WHEN f.promotion_key IS NULL
            THEN f.quantity_sold ELSE 0 END) AS base_units,
    -- Volume-weighted average price (never SUM unit_price)
    ROUND(SUM(CASE WHEN f.promotion_key IS NOT NULL
                THEN f.net_revenue ELSE 0 END)
            / NULLIF(SUM(CASE WHEN f.promotion_key IS NOT NULL
                THEN f.quantity_sold ELSE 0 END), 0), 2) AS
promo_avg_price,
    ROUND(SUM(CASE WHEN f.promotion_key IS NULL
                THEN f.net_revenue ELSE 0 END)
            / NULLIF(SUM(CASE WHEN f.promotion_key IS NULL
                THEN f.quantity_sold ELSE 0 END), 0), 2) AS
base_avg_price
FROM   fact_sales_transaction f
JOIN   dim_product    p    ON f.product_key = p.product_key
LEFT JOIN dim_promotion prm ON f.promotion_key = prm.promotion_key
GROUP BY p.product_name, prm.promotion_name
ORDER BY promo_revenue DESC;

-- =====
-- 3. CUSTOMER COHORT REVENUE CURVE
-- =====

```

```

WITH cohort_base AS (
  -- Identify each customer's cohort (first purchase fiscal month)
  SELECT
    customer_key,
    MIN(d.fiscal_year * 100 + d.fiscal_month) AS cohort_period
  FROM   fact_sales_transaction f
  JOIN   dim_date d ON f.date_key = d.date_key
  WHERE  f.customer_key <> -1          -- exclude Unknown Customer
  GROUP  BY customer_key
)
SELECT
  cb.cohort_period,
  (d.fiscal_year * 100 + d.fiscal_month) - cb.cohort_period AS
periods_since_acquisition,
  COUNT(DISTINCT f.customer_key)           AS active_customers,
  SUM(f.net_revenue)                       AS cohort_revenue
FROM   fact_sales_transaction f
JOIN   dim_date    d  ON f.date_key    = d.date_key
JOIN   cohort_base cb ON f.customer_key = cb.customer_key
WHERE  f.customer_key <> -1
GROUP  BY cb.cohort_period,
          (d.fiscal_year * 100 + d.fiscal_month) - cb.cohort_period
ORDER  BY cb.cohort_period,
          periods_since_acquisition;

```

Student

The model is built. Now show me the analytics queries that make it pay off - starting with sales.

Claude • AI Assistant

The test of a dimensional model is not its schema diagram - it is the quality and simplicity of the analytical SQL it enables. Well-designed dimensional models produce queries that read like English: `SELECT category_name, fiscal_year, SUM(net_revenue)` with two joins.

Badly designed models produce queries with twelve joins, CTEs to work around grain violations, and CASE statements to handle NULL FKs.

CONCEPT CARD

Revenue analysis patterns:

1. **Revenue by dimension** - GROUP BY any single dimension: `SELECT region, SUM(net_revenue)` . Works because net_revenue is additive. No SQL gymnastics needed.
2. **Year-over-year comparison** - use the date dimension's `fiscal_year` and `fiscal_month` columns: `SUM(CASE WHEN fiscal_year=2024 THEN revenue END) AS fy24, SUM(CASE WHEN fiscal_year=2023 THEN revenue END) AS fy23` .
3. **Rolling 30-day revenue** - use the pre-computed `is_last_30_days` flag from the date dimension: `WHERE d.is_last_30_days = 1` . No `DATEADD` logic needed.
4. **Revenue variance** - $(\text{FY24 revenue} - \text{FY23 revenue}) / \text{FY23 revenue} \times 100$ as a percentage. Always divide by `NULLIF(denominator, 0)` to handle periods with no prior-year data.

CONCEPT CARD

Trend analysis patterns:

1. **Moving average** - `AVG(net_revenue) OVER (PARTITION BY store_key ORDER BY date_key ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` - 7-day moving average by store.
2. **Cumulative revenue** - `SUM(net_revenue) OVER (PARTITION BY fiscal_year ORDER BY date_key)` - running total resets at each fiscal year start.
3. **Rank by revenue** - `RANK() OVER (PARTITION BY category_name ORDER BY SUM(net_revenue) DESC)` - top N stores or products per category.
4. **Period-over-period delta** - `LAG(monthly_revenue, 1) OVER (ORDER BY fiscal_year, fiscal_month)` - compare this month to last month.

CONCEPT CARD

Cohort analysis - the dimensional model advantage:

1. **What is a cohort?** - a group of customers who share a common starting event: 'customers who made their first purchase in Q1 FY2024.'
2. **Cohort identification** - `dim_customer.first_purchase_date` or `MIN(f.date_key) OVER (PARTITION BY f.customer_key)` from the fact table itself.
3. **Cohort revenue curve** - for each cohort month: total revenue in month 0 (acquisition month), month 1, month 2, etc. Reveals: how much revenue does a cohort generate over its lifetime?
4. **Cohort retention** - % of cohort customers who made at least one purchase in each subsequent month. Uses `COUNT(DISTINCT customer_key)` per cohort × period.

EXAMPLE CARD

Year-over-year revenue by product category:

sql — Code Example

```

SELECT
    p.category_name,
    SUM(CASE WHEN d.fiscal_year = 2024 THEN f.net_revenue ELSE 0 END) AS
fy24_revenue,
    SUM(CASE WHEN d.fiscal_year = 2023 THEN f.net_revenue ELSE 0 END) AS
fy23_revenue,
    ROUND(100.0 *
        (SUM(CASE WHEN d.fiscal_year = 2024 THEN f.net_revenue ELSE 0 END)
        - SUM(CASE WHEN d.fiscal_year = 2023 THEN f.net_revenue ELSE 0 END))
        / NULLIF(SUM(CASE WHEN d.fiscal_year = 2023
                        THEN f.net_revenue ELSE 0 END), 0), 1) AS
yoy_growth_pct
FROM    fact_sales_transaction f
JOIN    dim_product p ON f.product_key = p.product_key
JOIN    dim_date    d ON f.date_key    = d.date_key
WHERE   d.fiscal_year IN (2023, 2024)
GROUP  BY p.category_name
ORDER  BY fy24_revenue DESC;

```

🔗 ANALOGY

Sales analytics on a dimensional model is like **reading a well-indexed book**. The index (the dimension keys and attributes) lets you jump directly to any chapter (store, product, date, category).

A poorly designed model is like a book with no index - to find 'all sales in the Electronics category,' you read every page.

⚠️ WARNING

Window functions on non-additive measures produce wrong results.

`AVG(unit_price) OVER (...)` computes an unweighted average price - meaningless. Always apply window functions to additive measures or compute volume-weighted versions: `SUM(net_revenue) OVER (...) / NULLIF(SUM(quantity_sold) OVER (...), 0)` for a rolling weighted average price.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Joining the fact table to the date dimension ONLY by `date_key` and then computing `DATEADD(sale_date, -1, 'year')` for year-over-year in SQL. This bypasses the fiscal calendar in the date dimension - fiscal Q1 2024 may span different calendar months than fiscal Q1 2023.

Correct Way: Use `d.fiscal_year` and `d.fiscal_quarter` from the date dimension for all time-based grouping. The fiscal calendar is in the dimension; don't re-compute it in the query.

EXAM TIP CARD

Interview classic: "Write a query showing month-over-month revenue growth for the past 12 fiscal months." Winning answer: Use `dim_date.fiscal_year` and `dim_date.fiscal_month` for grouping. Compute monthly revenue in a CTE.

Use `LAG(monthly_revenue, 1)` for the prior month. Compute `(current - prior) / NULLIF(prior, 0) × 100`.

Filter by `relative_period_month BETWEEN -11 AND 0` from the date dimension.

🔥 EXERCISE

Write the following queries over the retail star schema: (a) top 5 products by net revenue for the last 30 days, (b) revenue by store region compared to the same period last fiscal year, (c) 7-day moving average revenue by product department, (d) daily revenue ranked within each store format (hypermarket, supermarket, express).

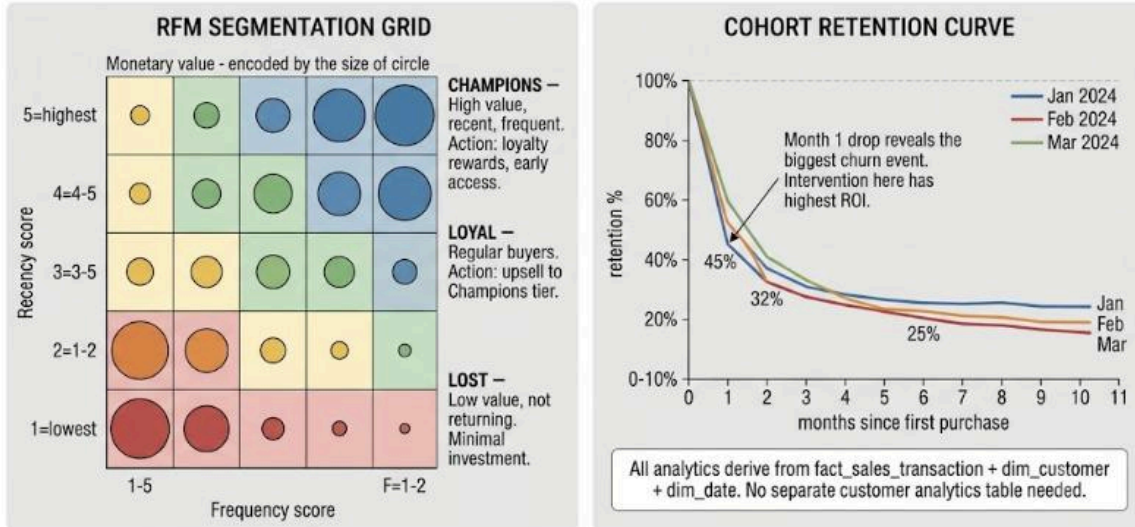
🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Write SQL for a complete sales analytics package over a retail star schema. Include: (1) YoY revenue by category with growth %, (2) rolling 7-day revenue by store, (3) revenue market share by store format within region, (4) promotion lift analysis (revenue per promoted product during vs outside promotion period), (5) a cohort revenue curve showing monthly revenue by customer acquisition cohort for the past 6 cohorts."

CUSTOMER ANALYTICS — RFM, RETENTION, AND LTV

Claude • AI Assistant

Customer Analytics – RFM Segmentation Grid and Retention Curve



The dimensional model enables both panels with one schema. The grain (one row per POS line item) is fine enough for all three RFM components.

Customer Analytics - RFM Segmentation Grid and Retention Curve

sql – RFM scoring, retention analysis, and LTV by acquisition channel

```
-- =====
-- 1. FULL RFM SEGMENTATION WITH LABELS
-- =====

WITH rfm_base AS (
    SELECT
        f.customer_key,
        MAX(f.date_key) AS last_purchase_key,
        COUNT(DISTINCT f.transaction_number) AS frequency,
        SUM(f.net_revenue) AS monetary
    FROM fact_sales_transaction f
    WHERE f.customer_key <> -1 -- exclude Unknown Customer
    AND f.correction_flag = 0
    GROUP BY f.customer_key
),
rfm_scored AS (
    SELECT
        customer_key, last_purchase_key, frequency, monetary,
        NTILE(5) OVER (ORDER BY last_purchase_key DESC) AS r,
        NTILE(5) OVER (ORDER BY frequency DESC) AS f,
        NTILE(5) OVER (ORDER BY monetary DESC) AS m
    FROM rfm_base
)
SELECT
    rs.customer_key,
    c.customer_name,
```



```

c.email,
c.loyalty_tier,
rs.monetary,
rs.frequency,
rs.r, rs.f, rs.m,
rs.r * 100 + rs.f * 10 + rs.m                AS rfm_score,
CASE
    WHEN rs.r >= 4 AND rs.f >= 4                THEN 'Champions'
    WHEN rs.r >= 3 AND rs.f >= 3                THEN 'Loyal'
    WHEN rs.r >= 4 AND rs.f < 3                THEN 'Promising'
    WHEN rs.r < 3 AND rs.f >= 4                THEN 'At Risk'
    WHEN rs.r < 2 AND rs.f < 2                THEN 'Lost'
    ELSE 'Needs Attention'
END                                            AS rfm_segment
FROM    rfm_scored        rs
JOIN    dim_customer      c  ON rs.customer_key = c.customer_key
        AND c.is_current = 1
ORDER  BY rs.monetary DESC;

-- =====
-- 2. MONTHLY COHORT RETENTION CURVE
-- =====

WITH cohort_month AS (
    SELECT customer_key,
           MIN(d.fiscal_year * 100 + d.fiscal_month) AS cohort_ym
    FROM    fact_sales_transaction f
    JOIN    dim_date d ON f.date_key = d.date_key
    WHERE   f.customer_key <> -1
    GROUP  BY f.customer_key
),
cohort_sizes AS (
    SELECT cohort_ym, COUNT(*) AS cohort_size FROM cohort_month GROUP BY
cohort_ym
)
SELECT
    cm.cohort_ym,
    (d.fiscal_year * 100 + d.fiscal_month) - cm.cohort_ym AS months_since,
    COUNT(DISTINCT f.customer_key)                        AS
active_customers,
    cs.cohort_size,
    ROUND(100.0 * COUNT(DISTINCT f.customer_key)
          / NULLIF(cs.cohort_size, 0), 1)                AS retention_pct
FROM    fact_sales_transaction f
JOIN    dim_date      d  ON f.date_key      = d.date_key
JOIN    cohort_month  cm ON f.customer_key = cm.customer_key
JOIN    cohort_sizes  cs ON cm.cohort_ym   = cs.cohort_ym

```

```

WHERE f.customer_key <> -1
GROUP BY cm.cohort_ym, (d.fiscal_year*100 + d.fiscal_month) - cm.cohort_ym,
cs.cohort_size
ORDER BY cm.cohort_ym, months_since;

-- =====
-- 3. LTV BY ACQUISITION CHANNEL WITH AOV
-- =====

SELECT
    c.acquisition_ch                                AS channel,
    COUNT(DISTINCT f.customer_key)                  AS customer_count,
    SUM(f.net_revenue)                               AS total_ltv,
    ROUND(SUM(f.net_revenue)
          / NULLIF(COUNT(DISTINCT f.customer_key), 0), 2) AS
avg_ltv_per_customer,
    ROUND(SUM(f.net_revenue)
          / NULLIF(COUNT(DISTINCT f.transaction_number),0), 2) AS
avg_order_value
FROM fact_sales_transaction f
JOIN dim_customer c ON f.customer_key = c.customer_key
                     AND c.is_current = 1
WHERE f.customer_key <> -1 AND f.correction_flag = 0
GROUP BY c.acquisition_ch
ORDER BY avg_ltv_per_customer DESC;

```

Student

Show me the customer analytics - RFM segmentation, retention, and lifetime value - using the same dimensional model.

Claude • AI Assistant

Customer analytics is where the dimensional model pays its largest dividend. The combination of **dim_customer** (demographic and loyalty attributes), **dim_date** (temporal context), and **fact_sales_transaction** (every purchase event) enables three of the most commercially valuable analytical frameworks: RFM segmentation for targeting, retention analysis for churn management, and LTV modeling for acquisition investment decisions.

CONCEPT CARD

RFM Segmentation - Recency, Frequency, Monetary:

1. **Recency (R)** - how recently did the customer last purchase? Computed from: `MAX(date_key)` per customer. Customers who bought recently are more likely to buy again.
2. **Frequency (F)** - how often do they buy? `COUNT(DISTINCT transaction_number)` per customer in the analysis window.
3. **Monetary (M)** - how much do they spend? `SUM(net_revenue)` per customer in the analysis window.
4. **Scoring** - each dimension is scored 1-5 (5=best).
Quintile scoring: `NTILE(5) OVER (ORDER BY recency DESC)` for R (higher recency = lower days-since-last-purchase = better). The combined RFM score (e.g., 555 = Champions, 111 = At-Risk Lost) drives targeted marketing actions.
5. **Dimensional model advantage** - all three components come from a single JOIN: fact → dimcustomer → dimdate. No complex pipeline needed.

CONCEPT CARD

Retention analysis:

1. **Cohort retention rate** - for a cohort of customers who first purchased in period C: what % purchased again in period C+1, C+2, C+N?
2. **Computation** - `COUNT(DISTINCT customer_key in period C+N who also exist in cohort C) / COUNT(DISTINCT customer_key in cohort C)` = retention rate.
3. **Churn definition** - a customer who has not purchased in the last 90 days (or domain-specific threshold) is churned.
4. **At-risk identification** - customers with Recency score 1 or 2 (haven't bought in a long time) AND Frequency score 4 or 5 (historically frequent buyers) are the highest-priority churn targets: high value, slipping away.
5. **Win-back campaigns** - `WHERE last_purchase_days BETWEEN 90 AND 180 AND lifetime_revenue > 10000` - high-value churned customers worth a targeted win-back offer.

CONCEPT CARD

Customer Lifetime Value (LTV):

1. **Historical LTV** - `SUM(net_revenue)` from a customer's first to most recent purchase. Descriptive; tells you what you've earned, not what you will earn.
2. **Predicted LTV** - models project future spend based on purchase frequency and monetary trend. Dimensional models provide the historical base; predictive models are built on top.
3. **Average order value (AOV)** - `SUM(net_revenue) / COUNT(DISTINCT transaction_number)` per customer. Non-additive - compute from additive components.

4. **LTV segmentation** - group customers into LTV tiers. 'Top 20% of customers generate 80% of revenue' (Pareto principle check). Use `NTILE(5) OVER (ORDER BY lifetime_revenue DESC)` - top quintile = high-LTV segment.
5. **LTV by acquisition channel** - `GROUP BY dim_customer.acquisition_channel` - which channel brings in the highest-LTV customers? Drives marketing budget allocation.

EXAMPLE CARD

RFM computation with quintile scoring:

sql – Code Example

```
WITH customer_rfm AS (
  SELECT
    f.customer_key,
    MAX(f.date_key) AS last_purchase_date_key,
    COUNT(DISTINCT f.transaction_number) AS frequency,
    SUM(f.net_revenue) AS monetary
  FROM fact_sales_transaction f
  WHERE f.customer_key <> -1
    AND f.correction_flag = 0
  GROUP BY f.customer_key
)
SELECT
  customer_key,
  monetary,
  frequency,
  last_purchase_date_key,
  NTILE(5) OVER (ORDER BY last_purchase_date_key DESC) AS r_score,
  NTILE(5) OVER (ORDER BY frequency DESC) AS f_score,
  NTILE(5) OVER (ORDER BY monetary DESC) AS m_score
FROM customer_rfm;
```

🔗 ANALOGY

RFM is like **a doctor's patient health score**. Recency = last check-up date (recent = healthy engagement).

Frequency = how often they come in (frequent = committed patient). Monetary = total medical spend (high = complex healthcare needs).

Score all three; prescribe different interventions for different score combinations.

Champions (555) get loyalty rewards.

At-Risk (211) get a win-back call.

⚠️ WARNING

LTV calculations that include the Unknown Customer (-1) will be wrong. Unknown customers are anonymous transactions - aggregating them as a 'customer' produces a single fictional customer with millions of purchases. Always filter `WHERE customer_key <> -1` (or `<> [your unknown surrogate]`) in all customer-level analytics.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Computing `AVG(net_revenue)` per customer as the 'average purchase value.' This is the average value of one purchase - not meaningful for segmentation if customers have very different purchase frequencies.

Correct Way: Compute Average Order Value = `SUM(net_revenue) / COUNT(DISTINCT transaction_number)` per customer. This correctly reflects the average basket size regardless of how often the customer shops.

EXAM TIP CARD

Interview classic: "How would you compute RFM scores for customer segmentation over a dimensional warehouse?" Winning answer: *"CTE on factsalestransaction grouped by customerkey - compute: MAX(datekey) for recency, COUNT(DISTINCT transactionnumber) for frequency, SUM(netrevenue) for monetary. Then outer query applies NTILE(5) window function on each metric (descending for all three).*

Combine R+F+M scores to create segment labels: `555='Champions', RFMscore < 222='At Risk'`. Filter `customerkey <> -1` (unknown customers).

Join to dim_customer for name and contact data for campaign activation."

🔥 EXERCISE

Write the retention analysis query for this scenario: customers who made their first purchase in January 2024 (the cohort). For each subsequent month (Feb, Mar, Apr through Dec 2024): what % of the January cohort made at least one purchase?

Show retention as a percentage, falling from 100% in month 0.

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Write a complete customer analytics SQL package for a retail dimensional warehouse. Include: (1) RFM segmentation with quintile scoring and segment label assignment (Champions, Loyal, At Risk, Lost), (2) monthly retention curve for the last 6 acquisition cohorts, (3) LTV by acquisition channel with AOV and purchase frequency, (4) churn risk identification (high-frequency customers with recent drop-off), (5) top 10% customers by LTV with their current loyalty tier and acquisition channel."

COMMON MODELING MISTAKES — THE INTERVIEW TRAPS

Claude • AI Assistant

Six Common Modeling Mistakes - Visual Pattern and Fix		
Mistake Name	What the Bad Schema Looks Like (Red)	Correct Schema (Green)
MIXED GRAIN	fact_sales with both header rows (NULL product_key, revenue=total) and line rows (product_key filled, revenue=line amount)	One grain per fact table. Aggregate to header at query time.
OVER-SNOWFLAKING	dim_product -> dim_subcategory -> dim_category -> dim_department (4 tables, 3 joins)	All four levels flattened into dim_product. GROUP BY department_name directly.
MISSING CONFORMED DIMS	Sales dim_product keys 1-5000 and Inventory dim_product keys 1-5000 from separate sequences - key collision	Single master dim_product sequence, built once before either fact table. Bus Matrix governance.
NON-ADDITIVE SUMMED	SUM(margin_pct) = 4200% result	Store gross_revenue + cogs. Compute margin_pct = SUM(rev-cost)/SUM(rev) at query time.
NATURAL KEYS AS FKs	fact(store_code VARCHAR, product_code VARCHAR) - breaks on source key renumber	fact(store_key INT, product_key INT) - surrogate keys via pipeline. Natural keys in dim only.
NULL FKs	customer_key=NULL for 40% of rows - dropped by inner JOIN	Unknown Customer surrogate=-1. No NULLs in fact FKs ever.

Each mistake has a characteristic wrong-result symptom. Knowing the symptom tells you where to look when a report is wrong.

Six Common Modeling Mistakes - Visual Pattern and Fix

Student

What are the mistakes that reveal a junior or untrained modeler - and that interviewers specifically look for?

Claude • AI Assistant

Six mistakes appear repeatedly in real warehouse designs and in design interview exercises. Each has a characteristic symptom - a query that returns wrong results - and a specific fix.

Knowing these patterns in both directions (mistake and correction) is the mark of a practitioner who has debugged real warehouse problems, not just read about them.

CONCEPT CARD

Mistake 1 - **Mixed grain in one fact table:**

1. **What it looks like** - a fact table that tries to capture both POS line items AND transaction header totals in the same table. Header rows have NULL productkey. *Line rows have a productkey.*

2. **The symptom** - `SUM(net_revenue)` on a query that JOINS product doubles or triples revenue. The header row's `net_revenue` (the transaction total) and each line item's `net_revenue` (the line total) both appear, and the sum double-counts.
3. **The fix** - one grain, one fact table. If you need header-level totals, aggregate the line-level fact at query time: `SUM(line_revenue) GROUP BY transaction_number`.
4. **The grain test** - read the grain statement; read every column. If any column cannot be described for every row in the table without qualification, you have mixed grain.

CONCEPT CARD**Mistake 2 - Over-snowflaking:**

1. **What it looks like** - a product dimension split into: `dim_product (product_key, subcategory_key) → dim_subcategory (subcategory_key, category_key) → dim_category (category_key, department_key) → dim_department`. Four tables, three extra joins.
2. **The symptom** - a query for 'revenue by department' requires four JOIN hops. BI tools fail to model the chain. Query performance degrades. Developers avoid the hierarchy and use hacks.
3. **The fix** - flatten all hierarchy levels into `dim_product : product_key, product_name, subcategory_name, category_name, department_name`. One table, zero extra joins. Redundancy is intentional and correct in dimensional modeling.
4. **The rule** - normalize dimensions only when the parent dimension is also used as a dimension of a separate fact table.

`dim_supplier` might have its own rows in the inventory fact - that's valid.

`dim_department` that exists only to join to `dim_category` that joins to `dim_product` is over-snowflaking.

CONCEPT CARD**Mistake 3 - Missing conformed dimensions:**

1. **What it looks like** - the Sales team's `dim_product` has surrogate keys 1–5000. The Inventory team's `dim_product` has surrogate keys 1–5000 from a DIFFERENT sequence. Both call the key `product_key`.

Productkey 4421 = 'Samsung TV 55' in Sales. Productkey 4421 = 'Nike Air Max' in Inventory.
2. **The symptom** - a JOIN between `fact_sales` and `fact_inventory` on `product_key` returns nonsense. Revenue for Samsung TV is mixed with inventory for Nike shoes. The result is silently wrong - no error raised.
3. **The fix** - Bus Matrix governance: establish `dim_product` with a single surrogate key sequence BEFORE building either fact table. Both teams reference the same master dimension.

CONCEPT CARD

Mistake 4 - Non-additive measure summed:

1. **What it looks like** - `profit_margin_pct DECIMAL(5,2)` is a column in the fact table. Someone writes `SELECT store_name, SUM(profit_margin_pct) FROM fact_sales JOIN dim_store GROUP BY store_name`.
2. **The symptom** - Store A reports a profit margin of 4,200%. The sum of 1,000 transaction-level margin percentages is meaningless.
3. **The fix** - store the additive components: `gross_revenue` and `cost_of_goods_sold`.

Compute margin at query time: `SUM(gross_revenue - cost_of_goods_sold) / NULLIF(SUM(gross_revenue), 0)`. If you must store margin in the fact, add `CONSTRAINT chk_no_summing CHECK(FALSE)` - just kidding.

Document and enforce through BI layer configuration.

CONCEPT CARD**Mistake 5 - Natural keys as fact foreign keys:**

1. **What it looks like** - `fact_sales(sale_date DATE, store_code VARCHAR(10), product_code VARCHAR(20), revenue DECIMAL)`. All FKs are source natural keys. No surrogate keys.
2. **The symptoms** - (a) source rennumbers store codes: every fact row must be updated. (b) Two source systems use the same `product_code` for different products: JOIN collision. (c) SCD Type 2 impossible: natural key is the same for all versions.
3. **The fix** - surrogate key pipeline. All fact FKs are integer surrogates. Natural keys live only in dimension tables and the key pipeline mapping table.

CONCEPT CARD**Mistake 6 - NULL foreign keys in fact tables:**

1. **What it looks like** - anonymous customer transactions have `customer_key = NULL` in `fact_sales_transaction`. About 40% of all transactions.
2. **The symptom** - `SELECT customer_key, SUM(revenue) FROM fact_sales JOIN dim_customer ... GROUP BY customer_key` drops all 40% of transactions. The revenue total is 40% too low. The error is silent - the query succeeds without a warning.
3. **The fix** - create an 'Unknown Customer' row in `dim_customer` with a well-known surrogate (e.g., -1). Load all anonymous transactions with `customer_key = -1`. Query now includes all rows. Filter `WHERE customer_key <> -1` when customer-specific analysis is needed.

🔗 ANALOGY

These six mistakes are like **six ways to fail a structural engineering exam**. Mixed grain = building a house that is simultaneously a bungalow and a three-story flat (structurally incoherent).

Over-snowflaking = requiring three hallways to reach the kitchen (unnecessary complexity).
Missing conformed dims = two wings of the same building using different measurement systems (metric vs imperial).

Non-additive summed = adding temperatures across rooms to get the 'total house temperature' (physically meaningless). Natural keys = naming every room after its current occupant and renaming it every time the occupant moves.

NULL FKs = rooms that don't appear on the floor plan but definitely exist.

EXAM TIP CARD

Interview classic: "A BI report shows revenue summing to 300% of the expected total. What do you look for?" Winning answer: "Three candidates: (1) Mixed grain - is the fact table accidentally summing both line-level and header-level revenue?

Check for rows where a key FK (like product_key) is NULL while revenue is non-zero. (2) Fan trap - is there a many-to-many join creating row multiplication?

Check the JOIN path. (3) Duplicate loads - did the ETL run twice?

Check loaddate distribution. Query correctionflag to see if correction rows account for it."

🔥 EXERCISE

Examine this fact table: `fact_orders(order_date DATE, store_code VARCHAR, product_code VARCHAR, customer_email VARCHAR, transaction_amount DECIMAL, line_amount DECIMAL, margin_pct DECIMAL)`. Identify every modeling mistake from the six categories above.

Rewrite the DDL correctly.

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Review this dimensional model design for a hospital: factencounter has columns patientname, doctorname, hospitalname, diagnosiscodes VARCHAR(200), totalcost, costperdiagnosis_pct. Identify every dimensional modeling mistake (grain, normalization, conformed dims, key types, non-additive measures, multi-valued attributes).

Rewrite the model correctly with proper fact and dimension tables."

MOCK DESIGN INTERVIEW — WHITEBOARD PROTOCOL AND FIVE QUESTIONS

Design Interview Whiteboard Protocol – Six Steps in Order

1. CLARIFY (2 min):

What is the finest grain available?
Batch or real-time?
Primary analytical questions?
Key stakeholders?

Never skip this.
Interviewers give extra credit for asking the right questions.

2. STATE BUSINESS PROCESS (30 sec):

The organizational activity being modeled.
Name it precisely: retail POS transactions, insurance claims, ride trips.

3. DECLARE THE GRAIN FIRST (1 min):

One row represents ONE [precise entity]. **ANNOUNCE IT.**
Write it on the board. Everything derives from this.

4. LIST DIMENSIONS (1 min):

Every FK. Every analytical lens.
Name role-playing dims separately (order_date, ship_date).
Name conformed dims explicitly.

5. CLASSIFY FACTS (1 min):

For each measure: Additive / Semi-additive / Non-additive.
For non-additive: name the additive components you will store instead.

6. HANDLE FOLLOW-UPS:

SCD type for each dim. Late-arriving data.
Volume scale (partition, aggregate layer).
Conformed dimension and Bus Matrix position.

Total: 8 minutes for a well-scoped question.
12 minutes for a complex multi-process question.
If still listing columns after 4 minutes, you went to physical design too early.

The grain statement is the single most important output. Say it first.
Write it on the board. Every column you list must be consistent with it.

Design Interview Whiteboard Protocol - Six Steps in Order

sql – Five interview question answers – key SQL patterns for each

```
-- =====
-- Q1: RIDE-HAILING WAREHOUSE
-- Grain: one row per completed trip
-- Non-additive trap: surge_multiplier – store components instead
-- =====
CREATE TABLE fact_trip (
    trip_date_key      INT NOT NULL,
    driver_key         INT NOT NULL,    -- FK → dim_driver (Type 2 SCD on
tier/rating)
    rider_key          INT NOT NULL,    -- FK → dim_rider
    pickup_zone_key    INT NOT NULL,    -- FK → dim_zone (role-playing:
pickup)
    dropoff_zone_key   INT NOT NULL,    -- FK → dim_zone (role-playing:
dropoff)
    trip_id            VARCHAR(50),     -- DEGENERATE DIMENSION
    trip_distance_km    DECIMAL(8,2),   -- additive
    trip_duration_min   INT,             -- additive
```

```

        base_fare          DECIMAL(10,2), -- additive
        surge_amount       DECIMAL(10,2), -- additive (surge_multiplier ×
base_fare - base_fare)
        total_fare         DECIMAL(10,2), -- additive
        driver_payout      DECIMAL(10,2), -- additive
        platform_fee       DECIMAL(10,2)  -- additive
        -- surge_multiplier NOT stored – non-additive
        -- derive at query: SUM(total_fare)/NULLIF(SUM(base_fare),0) = weighted
avg multiplier
    );

-- =====
-- Q3: SCD TYPE DECISION – POINT-IN-TIME DIMENSION LOOKUP
-- Which version of the customer was active at transaction time?
-- =====
SELECT
    f.date_key,
    c.loyalty_tier,          -- the tier the customer had AT TIME OF PURCHASE
    SUM(f.net_revenue) AS revenue
FROM    fact_sales_transaction f
JOIN    dim_customer c
    ON   c.customer_key    = f.customer_key
    -- No is_current filter – we want the version active at purchase time
    -- NOTE: this only works if the fact table FK points to the versioned
surrogate
    -- which it does if the ETL resolved to is_current=1 at load time
GROUP BY f.date_key, c.loyalty_tier;

-- =====
-- Q4: CONFORMED DIMENSION – CROSS-MART DRILL-ACROSS
-- =====
-- Retail Sales Mart: fact_sales + dim_product (same keys)
-- Inventory Mart:    fact_inventory + dim_product (same keys)
-- Works because both use the same master dim_product surrogate sequence
SELECT
    p.category_name,
    SUM(s.net_revenue)    AS monthly_revenue,
    SUM(i.units_on_hand)  AS end_month_inventory
FROM    dim_product p
JOIN    fact_sales_transaction s ON s.product_key = p.product_key
JOIN    fact_inventory_snapshot i ON i.product_key = p.product_key --
CONFORMED
GROUP BY p.category_name;

-- =====
-- Q5: SCALE – PRE-AGGREGATED SUMMARY FACT FOR HIGH VOLUME

```

```
-- At 10B rows/day, pre-aggregate to daily productxstore summary
-- =====
CREATE TABLE fact_daily_sales_summary (
    date_key          INT NOT NULL,          -- already at day grain – no hour
needed
    store_key         INT NOT NULL,
    product_key       INT NOT NULL,
    total_revenue     DECIMAL(14,2),
    total_units       INT,
    transaction_cnt   INT,
    PRIMARY KEY (date_key, store_key, product_key)
);
-- Populate nightly from fact_sales_transaction:
INSERT INTO fact_daily_sales_summary
SELECT
    date_key, store_key, product_key,
    SUM(net_revenue), SUM(quantity_sold), COUNT(DISTINCT transaction_number)
FROM fact_sales_transaction
WHERE date_key = :yesterday
AND correction_flag = 0
GROUP BY date_key, store_key, product_key
ON CONFLICT (date_key, store_key, product_key) DO UPDATE
    SET total_revenue = EXCLUDED.total_revenue,
        total_units   = EXCLUDED.total_units,
        transaction_cnt = EXCLUDED.transaction_cnt;
```

Student

How do I structure a dimensional design interview answer? What is the verbal protocol, and what are the most likely questions?

Claude • AI Assistant

The design interview tests two distinct skills: dimensional modeling knowledge (do you know the concepts?) and design communication skill (can you think through a novel problem live, articulate your reasoning, and defend your choices?). Most candidates have the knowledge.

The distinguishing skill is the verbal protocol - the structured way of working through a design on the whiteboard that shows the interviewer your thought process is organized.

CONCEPT CARD

The whiteboard verbal protocol - six steps:

1. **Clarify before designing** - 'Before I start, can I ask a few questions? What is the finest grain of data available? Is this real-time or batch? What are the primary analytical questions it needs to answer?'
2. **State the business process** - 'I'll model the [process]. The organizational activity I'm capturing is [description].'
3. **Declare the grain first, loudly** - 'The grain is: one row represents [exact statement]. This is the most important decision and everything else derives from it.'
4. **List dimensions by analytical role** - 'The dimensions are: [list]. These are the lenses analysts will use to filter and group.'
5. **List and classify facts** - 'The measures are: [list]. [X] is additive - we can SUM across any dimension. [Y] is semi-additive - cannot SUM across time. [Z] is non-additive - I'll store the additive components instead.'
6. **Handle the interviewer's follow-up** - anticipate: SCD type choice, conformed dimension question, late-arriving data, volume/scale considerations. Have a structured answer for each.

CONCEPT CARD

Five most common design interview questions:

1. **'Design a data warehouse for a ride-hailing company (Uber/Ola).'** - Business processes: Trip Completion, Driver Earnings, Surge Pricing Events, Rider Complaints.

Key grain: one row per completed trip leg. Key dimension: *dimdriver*, *dimrider* (both Type 2 SCD on tier/rating), *dim_zone* (pickup/dropoff as role-playing geography).

Key traps: accumulating snapshot for trip lifecycle (requested → accepted → in-progress → completed → rated). Non-additive: *surgemultiplier* (*store basefare* and *total_fare* separately).

2. **'Design a warehouse for an e-commerce company (Amazon/Flipkart).'** - Covered in Chapter 3. Key trap: clickstream grain vs order grain - separate fact tables. Cart abandonment as near-factless fact.

3. **'How do you handle slowly changing dimensions?'** - Three types.

Type 1: overwrite - no history (*acquisition_channel*). Type 2: new row - full history (*address*, *loyalty tier*).

Type 3: prior value column - limited history (for gradual rollout). Decision matrix: Type 2 for anything where 'what was the value at the time of the transaction?' matters.

4. **'What is a conformed dimension? Give an example.'** - Covered in Chapter 3. Key points: same surrogate key, same attribute definitions, same grain across all marts that use it. Example: *dim_product* used by both Sales and Inventory marts - enables drill-across by product.

5. 'What would you do differently if the data volume is 10 billion rows per day?' -

Partitioning (range by date_key), columnar storage format (Parquet, ORC), separate aggregation layers (daily/weekly pre-aggregated fact tables), appropriate grain for summary mart (daily product × store summary instead of individual transactions), streaming ingestion (Kafka → Delta Lake) for real-time requirements.

CONCEPT CARD

How to handle 'I don't know' moments in the interview:

1. **Never fake confidence** - experienced interviewers catch this and it damages trust more than the knowledge gap.
2. **Acknowledge and bound** - 'I haven't implemented this specific scenario, but based on the principle of [relevant concept], I would approach it by [reasoned extension].'
3. **Think aloud** - 'Let me reason through this. The problem is [X]. The dimensional modeling principle that applies is [Y]. That would suggest [Z].'
4. **Ask clarifying questions** - sometimes 'I don't know' is actually 'I need more information.' Clarifying questions buy thinking time and show structured reasoning.
5. **Close the loop** - if you reasoned to an answer: 'Does that approach align with what you're looking for?' Invites the interviewer to correct or confirm - showing intellectual humility.

🔗 ANALOGY

A design interview is like a **live surgery simulation in medical school**. The examiner doesn't just want to see you operate correctly - they want to see you announce each step, explain your reasoning, handle a complication, and remain methodical under pressure.

A surgeon who operates silently and then explains afterward gives the examiner nothing to grade. Narrate the design as you do it.

⚠️ WARNING

The most common interview failure is jumping to DDL before declaring the grain.

Candidates who immediately start listing columns before writing the grain statement signal that they design by instinct, not by process. Interviewers who know dimensional modeling will ask 'what is the grain?' and the answer must come immediately and precisely.

🚫 COMMON PITFALL / MISTAKE

Incorrect Way (in the interview): 'I would create a fact table with columns for date, store, product, customer, revenue, units, margin...' - jumping to columns without stating the grain or classifying measures.

Correct Way: 'The grain is one row per POS line item. The dimensions are [list].'

The measures are [list], of which revenue and units are additive, and margin percent is non-additive - so I would store revenue and cost as separate additive columns and derive margin at query time.'

EXAM TIP CARD

Universal interview closing move - the Bus Matrix: *Whenever a design interview question involves multiple business processes, always close with: 'If we zoom out, here is where this fact table sits in the Bus Matrix alongside the other processes we'd need to model. [List the other business processes.] The dimensions that appear in multiple rows are the conformed dimensions we'd build first, in this order...'* This shows enterprise thinking, not just single-table design.

🔥 EXERCISE

Practice this question out loud, timed at 8 minutes: 'Design a data warehouse for a subscription streaming service (Netflix/Hotstar).' Work through: clarifying questions (2 minutes), business processes and grain statements (2 minutes), dimensions and facts (2 minutes), SCD decisions and one conformed dimension example (1 minute), Bus Matrix sketch (1 minute). Record yourself.

Play it back. Grade: did you state the grain first?

Did you classify the measures? Did you name at least one conformed dimension?

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *"You are a senior data engineer interviewing a candidate for a data warehousing role. Conduct a 10-minute mock dimensional design interview for a healthcare insurance company.*

Ask: the main design question (design a claims warehouse), then 4 follow-up questions probing: SCD type choice for provider dimension, how they handle a claim with multiple procedures, what they would do if claims arrive before provider records, and how they would validate data quality before loading. Evaluate each answer against dimensional modeling best practices."

CHAPTER REVIEW & SUMMARY AND REFCARD CONCLUSION

Claude • AI Assistant

CHAPTER SUMMARY

This chapter synthesized all five weeks of Week 3 content into a design workflow, analytics queries, mistake pattern recognition, and interview performance.

- **END-TO-END DESIGN:** The four-step Kimball process - (1) select the business process, (2) declare the grain, (3) identify dimensions, (4) identify and classify facts. Apply it in order for every new fact table. Physical decisions follow logical decisions - surrogate keys, SCD types, partition strategy, correction metadata. Requirements before DDL. Grain before columns. Bus Matrix before any table is built.
- **SALES ANALYTICS:** Additive measures enable direct GROUP BY aggregation. Year-over-year: conditional aggregation on *fiscalyear from dimdate*. Moving average: window function ROWS BETWEEN on fact, additive measures only. Cohort analysis: MIN(datekey) *per customerkey* as cohort anchor; subsequent periods as integer offset from cohort period. Promotion lift: CASE WHEN promotion_key IS NOT NULL partitions promoted vs baseline facts in one scan.
- **CUSTOMER ANALYTICS:** RFM = MAX(datekey) *for Recency*, COUNT(DISTINCT transactionnumber) *for Frequency*, SUM(netrevenue) *for Monetary*. NTILE(5) window function scores each dimension. Retention = cohortsize baseline then COUNT(DISTINCT customerkey) *per subsequent period, divided*. LTV by channel = SUM(revenue)/COUNT(distinct customers) *per acquisitionchannel*. Always filter customerkey < > -1 *for all customer analytics*. AOV = SUM(revenue)/COUNT(DISTINCT transactionnumber) - never AVG(unit_price).
- **COMMON MISTAKES:** Six canonical errors - mixed grain (symptom: doubled revenue), over-snowflaking (symptom: multi-join hierarchy), missing conformed dims (symptom: cross-mart join collision), non-additive summed (symptom: impossible percentages), natural keys as FKs (symptom: breaks on source renumber), NULL FKs (symptom: silent revenue drop in JOINS). Every mistake has a characteristic symptom and a one-sentence fix.
- **MOCK DESIGN INTERVIEW:** Six-step verbal protocol - clarify, state process, declare grain (loudly and first), list dimensions, classify facts, handle follow-ups. Five common questions: ride-hailing, e-commerce, SCD types, conformed dimensions, volume/scale. The grain statement is the single most evaluated element - state it first, state it precisely, write it on the board. Close every multi-process design with Bus Matrix positioning.

Claude • AI Assistant

The Refcard is complete. You now hold the full Week 3 dimensional modeling toolkit - from grain declarations and fact table classification through advanced dimension patterns, enterprise Bus Matrix design, multi-domain worked examples, production ETL mechanics, and the interview fluency to articulate it all under pressure.

The concepts in these pages - grain, conformance, additivity, SCD, the bridge table, the Bus Matrix - are not abstract theory. They are the decisions that every production data warehouse encodes.

When you next encounter a wrong revenue figure in a BI report, a slowly changing customer attribute, or a cross-domain analytical question that 'should be simple,' you now know exactly which modeling decision failed and exactly how to fix it.